Solver Description

Rens Versnel ✉
Utrecht University, The Netherlands

Abstract

Submission to the exact track of the PACE challenge 2026. This solver consists of two parts: a depth-bounded search akin to existing FPT algorithms [3] for this problem, and a depth-first search procedure that iterates all agreement forests of the instance using incompatible triple and path pair constraints.

2012 ACM Subject Classification Theory of computation → Parameterized complexity and exact algorithms

Keywords and phrases Maximum Agreement Forest, PACE Challenge

1 Short algorithm description

The main algorithm starts by applying common cherry reduction, single-vertex tree reduction , reduction rule 2.2.1 (from [3]), reduction rule 1 (from [3]), subtree cluster reduction and LSI cluster reduction 2; all exhaustively. Then on each reduced cluster it applies whichever of the two algorithms (AF-DFS and HP-DBS) it deems the most able to solve it, often running AF-DFS for a short period before defaulting to HP-DBS. The AF-DFS algorithm iterates the agreement forests of an instance, by starting at the trivial solution (only single-vertex trees) and then walking through the set of agreement forests while incrementally checking the possible additions against pre-calculated constraints. This makes it very fast if an instance has few agreement forests (which is usually the case when the MAF is very large compared to the number of leaves). The HP-DBS algorithm is more like traditional FPT algorithms, except that it stores a set of 'hitting pairs' as constraints, meaning that for each pair one of the two must become a single-vertex tree. This set of hitting pairs helps formulate a few reduction rules and helps in choosing the right branching rule to apply.

Now we will describe the two algorithms separately:

1.1 Agreement forest depth-first search (AF-DFS)

There are instances of MAF where the optimal solution has an order nearly as high as the number of leaves; these instances are very hard to solve with depth-bounded search algorithms, but should actually not be that hard because there are very few agreement forests that need to be considered. To cover these cases, we introduce a new algorithm that essentially iterates all possible agreement forests in depth-first search manner, starting with the trivial solution consisting of only SVTs.

The first step is to define a canonical representation of each agreement forest and describe a DFS tree visiting all representations. Then, we define two operations on the canonical representations to enumerate all possible representations:

▶ **Definition 1** (Canonical representation (CANAF)). *Let A be an agreement forest of an instance Ω. For each component of A with at least 2 labels, list its labels in increasing order. Then order these lists in increasing order by their first (smallest) label. The resulting ordered collection $S_1, S_2, \ldots, S_m$ is the* canonical representation *of A (CANAF(A)).*

▶ **Definition 2** (Extend last component (EXTENDLASTCOMP)). *Let $S_1, S_2, \ldots, S_m$ be a canonical representation of an agreement forest A of instance Ω. The procedure EXTENDLASTCOMP(A, Ω) iterates all canonical representations created by appending an 'unused' label from Ω, that is*

Procedure *ITER-AF-DFS*(Ω, A)
  Input: A set $\Omega = \{F_1, \ldots, F_t\}$ of X-forests and an agreement forest A of instance Ω.
  Output: A set of agreement forests of Ω reachable from A by any combination of ExtendLastComp and AddNewComp operations.
  $\mathcal{A} \leftarrow \bigcup_{A' \in \textsc{ExtendLastComp}(A)} \text{ITER-AF-DFS}(\Omega, A')$
  $\mathcal{B} \leftarrow \bigcup_{A' \in \textsc{AddNewComp}(A)} \text{ITER-AF-DFS}(\Omega, A')$
  return $\{A\} \cup \mathcal{A} \cup \mathcal{B}$

Procedure *AF-DFS*(Ω)
  Input: A set $\Omega = \{F_1, \ldots, F_t\}$ of X-forests.
  Output: An minimum order agreement forest for Ω.
  $A \leftarrow$ the trivial agreement forest of Ω, consisting of SVTs
  $\mathcal{A} \leftarrow \text{ITER-AF-DFS}(\Omega, A)$
  return *the agreement forest in* $\mathcal{A}$ *with minimum order*

Figure 1 An exact algorithm for computing a MAF

larger than all labels in S_m*, to* S_m *and returns the subset of these that is an agreement forest of* Ω*.*

▶ **Definition 3** (Add new component (AddNewComp))**.** *Let* $S_1, S_2, \ldots, S_m$ *be a canonical representation of an agreement forest* A *of instance* Ω*. The procedure* AddNewComp(A, Ω) *iterates all canonical representations created by selecting two 'unused' labels* a, b *from* Ω *with* $a < b < \min S_m$ *and adding* $\{a, b\}$ *as a new component and returns the subset of these that is an agreement forest of* Ω*.*

It should be clear that *any* (maximum) agreement forest of an instance is reachable by a combination of ExtendLastComp and AddNewComp operations. It is also clear that calculating the order of an agreement forest from its canonical representation is also straightforward, and that we can recognize which canonical representation was a MAF among the iterated solutions.

Now we are ready to formulate the depth-first search routine iterating all agreement forests, see Figure 1; its correctness should be clear from the previous observations.

Determining if an X-forest is an agreement forest

To complete the description, we need to describe how we determine if an X-forest or its canonical representation is an agreement forest for an instance Ω (which we use in the ExtendLastComp and AddNewComp operations).

The approach is based on an ILP formulation for the rSPR distance problem, and thus for Maf on two binary trees, by Wu [4], which we extended to solve Maf on multiple binary trees. Wu defines two types of constraints, incompatible triple constraints and incompatible path constraints, which we will describe a modified version of:

▶ **Definition 4** (Incompatible triple)**.** *Let* Ω *be an instance of* Maf *and let* a, b *and* c *be three distinct labels from* Ω*. We say triple* $\{a, b, c\}$ *is an* incompatible triple *if there is a pair of forests* F_a*,* F_b *in* Ω *in which* a, b *and* c *are in the same component and for which the subgraphs induced by* $\{a, b, c\}$ *are not the same.*

▶ **Definition 5** (Incompatible path pair). *Let Ω be an instance of MAF and let a, b, c and d be four distinct labels from Ω. We say the collection $\{\{a, b\}, \{c, d\}\}$ is an* incompatible path pair *if there is a forest F in Ω in which a and b are connected and c and d are connected and in which the path between a and b shares an edge with the path between c and d.*

Then we can recognize agreement forests as follows:

▶ **Lemma 6.** *An X-forest A on the label-set of an instance Ω is an agreement forest for Ω if and only if:*

1. *no incompatible triple of Ω has all three labels in the same component in A, and*
2. *there is no incompatible path pair $\{\{a, b\}, \{c, d\}\}$ where a and b are connected and c and d are connected in A, and a and c are not connected in A.*

The proof of this lemma is tedious for some definitions of an agreement forest, but for this lemma we look at the definition often used in literature, based on all induced subtrees of the components being equal and all minimal rooted subtrees of the components being edge-disjoint in every tree. This definition corresponds one to one to the two types of constraints: by a well known lemma by Bordewich and Semple [2] two trees are equivalent if every triple of labels is equivalent; and by definition iff a forest contains an incompatible path pair, the minimal rooted subtrees of the components share an edge.

Method 2: enumerating valid components When the number of possible components satisfying the incompatible triple constraints is small, we use a different strategy to determine if a component satisfies the incompatible triple constraints: We create a DP table containing for each component A_i (as a canonical representation) all extension labels a such that putting a at the end of A_i results in a valid component w.r.t the incompatible triple constraints (and w.r.t the canonical representation constraints i.e., a larger that all labels in A_i).

To create this table we start off with a set of *compatible* triples C_3 and for each compatible triple (a, b, c) we iterate each label d that is larger than c. For each d, we check if all subsets of size 3 of (a, b, c, d) are compatible triples and if so we add d to $E((a, b, c))$ and we add (a, b, c, d) to C_4. Now C_4 contains all compatible quads and we can perform the same steps to calculate the possible extensions for this set, and so on. The result is a map E from valid components to valid extensions w.r.t incompatible triple constraints.

The proof that this is valid is simple: let C_i be a such a set with a component D that is not valid and where i is minimum, it must contain an incompatible triple, and thus has some subset in $C_{(i-1)}$ that contains it, contradicting the minimality of i.

In our solver we use this method to store the possible 'extensions' of all valid components of sizes 3 to 8 (incl). However, we only calculate this DP-table if none of the sets C_i exceed 300'000 items, due to possible time constraints and memory constraints.

1.2 Hitting pair depth-bounded search (HP-DBS)

This depth-bounded search algorithm is similar to the FPT algorithm by Shi et al [3], but we extend it to include hitting pair constraints:

▶ **Definition 7** (Hitting Pair). *Let X be a set of labels. A hitting pair set H is a set of pairs of two different labels, i.e., $H \subseteq (X \times X) \setminus \{\{x, x\} | x \in X\}$.*

The idea of a hitting pair $\{a, b\}$ is that it forces at least one of a and b to be a single vertex tree in the current execution branch. This is a hard constraint in the recursion, making a few more reduction rules possible.

The common cherry reduction and single vertex tree reduction rules are applied exhaustively between all other branching or reduction rules. The common cherry reduction is modified to adhere to the hitting pair constraints, only merging pairs of labels that are not in a hitting pair together. We will call these modified single-vertex tree reduction and common cherry reduction HP-RR1 and HP-RR2 respectively and we will say an instance is $RR1$-reduced (respectively $RR2$-reduced) if HP-RR1 (resp. HP-RR2) is applied exhaustively.

The introduction of hitting pairs also allows us to formulate a modified version of the base branching rule 2.1 as defined in [3]. Let (Ω, H, k) be an $RR1$- and $RR2$-reduced instance and let $\{a, b\}$ be a cherry in some forest in Ω where $\{a, b\} \notin H$.

▶ **Definition 8** (HP-Branching Rule 1 (HP-BR1)). *Branch in two ways:*
- *Remove all subtrees pendant to the path between a and b in all forests;*
- *Add $\{a, b\}$ to H.*

It is clear that this branching rule is safe from the proof of branching rule 2.1 in [3].

The previous branching rule is not applicable when all cherries in the instance are already a hitting pair. To complement this we introduce another branching rule that uses these hitting pairs.

Let (Ω, H, k) be an $RR1$- and $RR2$-reduced instance and let a be a label that is in at least one hitting pair and let B be the set of labels that is in a hitting pair with a (i.e., $B = \{b | \{a, b\} \in H\}$).

▶ **Definition 9** (HP-Branching Rule 2 (HP-BR2)). *Branch in two ways:*
- *Make a a single vertex tree in all forests;*
- *Make all labels in B single vertex trees in all forests.*

Again, the correctness should be clear: in an instance where a, b form a hitting pair, one of the two MUST become a single-vertex tree.

Despite the introduction of hitting pairs, reduction rule 2.2.1 from [3], which can have a great impact in some cases, is still applicable in most cases with some modifications:

▶ **Definition 10** (HP-Reduction Rule 3). *Let (Ω, H, k) be an $RR1$- and $RR2$-reduced instance and a and b be two labels that form a cherry in some forest and are not in any hitting pairs in H (i.e., $\nexists c$, s.t. $\{a, c\} \in H \vee \{b, c\} \in H$). If in every forest:*
- *a and b are either a cherry or separated by only one pendant subtree from becoming a cherry (i.e., there are 0 or 1 pendant subtrees on the path between a and b);*
- *and the label-sets of this pendant subtree are the same across all forests,*

then cut the pendant subtree between a and b (if applicable) in each forest, thereby making a and b a cherry in each forest.

The proof of this reduction rule is also a slightly modified version of the proof given Lemma 4.4 in [3], which we will not include here due to the page limit. However, it should not be difficult to verify that their proof still holds when neither a nor b is in a hitting pair.

To generate more hitting pairs, a new reduction rule is introduced. The idea behind it is the same as a well-known branching rule that states that if labels a and b form a cherry in one forest and are in different components in another, then (at least) one of them must become a single vertex tree. The correctness of this rule should therefore be clear enough.

▶ **Definition 11** (HP-Reduction Rule 4 (HP-RR4)). *Let (Ω, H, k) be an $RR1$- and $RR2$-reduced instance. If two labels a and b form a cherry in some forest and are in different components in another forest, then add hitting pair $\{a, b\}$ to H.*

HP-DBS(Ω, H, k)

Input: A set $\Omega = \{F_1, \ldots, F_t\}$ of X-forests and a set $H \subseteq (X \times X) \setminus \{\{x, x\} | x \in X\}$ of hitting pairs and a parameter k.

Output: An agreement forest for Ω of order at most k, in which for each hitting pair $\{a, b\} \in H$ either a is a single vertex tree or b is, or both, or report that no such agreement forest exists.

1 **if** $\textsc{ord}(\Omega) > k$ **then** return "no";
2 **if** Ω *contains no cherries* **then** unmerge all previously merged labels and return F_1;
3 apply the first applicable rule from HP-RR1, HP-RR2, and HP-RR3 and recurse;
4 exhaustively apply HP-RR4;
5 **if** *HP-RR5 is applicable* **then** apply it to obtain (Ω', H') and return HP-DBS(Ω', H', k);
6 **if** *Either form of cluster reduction is applicable* **then** apply it and solve the clusters;
7 apply an applicable branching rule from HP-BR1 and HP-BR2 and on each produced branch (Ω', H'), recursively call HP-DBS(Ω', H', k), returning any agreement forest found;

Figure 2 An exact algorithm for HP-Maf (Ω, H, k)

The introduction of hitting pairs also allows for the formulation of another reduction rule:

▶ **Definition 12** (HP-Reduction Rule 5 (HP-RR5)). *Let (Ω, H, k) be an RR1- and RR2-reduced instance. Let C be the label-set of some component of a forest in Ω with $|C| \geq 2$. If a is a label in C and for every label $c \in C \setminus \{a\}$, $\{a, c\}$ is a hitting pair in H, then make label a a single vertex tree in each forest in Ω.*

The safety of this reduction rule is again clear: if a is not a SVT, then all others in C must be, which leaves a to be the only leaf left in the component C.

Exact algorithm for Maf

The HP-DBS algorithm for solving HP-Maf is described in Figure 2. It should be clear that HP-DBS is also an exact algorithm Maf: any instance of (Ω, k) of Maf can be solved by solving HP-Maf $(\Omega, \emptyset, k)$ instead.

2 Cluster reduction

We use two types of cluster reduction in this solver: we will call the first type *LSI cluster reduction*. It is applicable when each forest in an instance contains a component with the same label-set; in this case we can simply extract that label set and solve it separately. We will call the second type *subtree cluster reduction*. It is applicable when each forest has a subtree with the same label-set; in this case we can cut the subtree and solve it separately, but we need to add dummy labels to determine if an agreement forest can bridge this edge that was just cut. Kelk recently posted a write-up [1] on this including sketch proofs.

References

1 Steven Kelk. A pedagogical implementation of the cluster reduction, 2026. URL: https://skelk.sdf-eu.org/cluster/.

2 Charles Semple, Mike Steel, et al. *Phylogenetics*, volume 24. Oxford University Press on Demand, 2003.

3 Feng Shi, Jianer Chen, Qilong Feng, and Jianxin Wang. A parameterized algorithm for the maximum agreement forest problem on multiple rooted multifurcating trees. *Journal of Computer and System Sciences*, 97:28–44, 2018. doi:10.1016/j.jcss.2018.03.002.

4 Yufeng Wu. A practical method for exact computation of subtree prune and regraft distance. *Bioinformatics*, 25(2):190–196, 2009. doi:10.1093/bioinformatics/btn606.